

→ Create a Linked list with the node having data part and pointer part where data is a character and pointer points to its next node, now accept a string where character part of each string is stored as data part in the node. Now find out if the linked list is palindrome or not.

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
    char data;
```

```
    struct node * next;
```

```
};
```

```
struct node * CreateNode(char ch) {
```

```
    struct node * newNode = (struct node *) malloc(sizeof(
        struct node));
```

```
    newNode->data = ch;
```

```
    newNode->next = NULL;
```

```
    return newNode;
```

```
}
```

```
int isPalindrome(struct node * head, int length) {
```

```
    char str[100];
```

```
    int i = 0;
```

```
    while (head != NULL) {
```

```
        str[i++] = head->data;
```

```
        head = head->next;
```

```
    }
```

```
    for (i = 0; i < length/2; i++) {
```

```
        if (str[i] != str[length-i-1]) {
```

```
            return 0;
```

```
        }
```

```
    }
    return 1;
```

```

int main() {
    struct node * head = NULL, * temp = NULL, * newnode;
    char str[100];
    int i, length;
    printf("Enter a string:");
    scanf("%s", &str);
    length = strlen(str);
    for (i = 0; i < length; i++) {
        newnode = createNode(str[i]);
        if (head == NULL) {
            head = newnode;
            temp = newnode;
        }
        else {
            temp->next = newnode;
            temp = newnode;
        }
    }
    printf("Linked list:");
    temp = head;
    while (temp != NULL) {
        printf("%c -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
    if (isPalindrome(head, length))
        printf("The linked list is a palindrome.\n");
    else
        printf("The linked list is not a palindrome.\n");
    return 0;
}

```